# Methods Overview

## Methods Overview I

The project is an eleven-stage, file-system-backed pipeline. `method s_pipeline.yaml` is the executable contract: each stage names its method, dependencies, inputs, outputs, definition of done, and stable failure code. Business logic lives in tested `src/` modules; numbered scripts only perform bootstrapping, argument parsing, orchestration, and boundary I/O.

## Pipeline stages I

1. **Multi-phase retrieval** (`01_multi_phase_search.py`) acquires records or runs the labelled offline fixture, applies configured filters, de-duplicates identifiers, and writes phase corpora plus `phase_metadata.json` and the `phase_artifact_manifest.json` phase-to-artifact provenance map.
2. **Corpus analysis** (`02_meta_analysis_pipeline.py`) computes subfields, temporal trends, TF-IDF/topics, and citation-network artifacts.
3. **Knowledge graph construction** (`03_build_knowledge_graph.py`) optionally extracts assertions and phase-aware hypothesis evidence; disabled or unmeasured LLM outputs remain explicitly pending.
4. **Visualization** (`04_generate_figures.py`) renders figures from analysis artifacts and records each figure's label, caption, filename, and generating stage in the figure registry.
5. **Manuscript hydration** (`05_inject_variables.py`) computes variables from outputs and writes rendered Markdown without changing source sections.
6. **Full-text assessment** (`06_fulltext_assessment.py`) records abstract, open-access, and PDF availability; downloading is opt-in and network-gated.
7. **Literature evaluation** (`07_literature_evaluation.py`) applies configured quality and fixture-honesty checks.

# Pipeline stages II

8. **Research dispatch** (`08_deep_research_dispatch.py`) replays a recorded report by default and exposes live provider dispatch only as an explicit opt-in.
9. **Bibliography export** (`09_export_bibliography.py`) writes a deterministic bibliography from the retained corpus and source references.
10. **Reproducibility assessment** (`10_reproducibility_assessment.py`) records workflow and source-consumption evidence.
11. **Publication validation** (`11_validate_outputs.py`) validates artifact, evidence, figure, and report contracts before the shared publication audit runs.

## Reproducibility model I

The offline path uses a deterministic synthetic fixture with reserved identifiers and generated authors. It is useful for CI and structural testing only; it is never a substitute for a representative live review. A live run must preserve the retrieval report, engine status, source identifiers, configuration, and claim classifications. Seeds and timestamp-free serializers are used where the pipeline supports exact replay.

## Configuration surface I

`manuscript/config.yaml` owns search terms, engines, phase boundaries, filters, sampling seeds, full-text policy, embedding settings, knowledge-graph settings, hypotheses, and subfield taxonomy. `domain_profile.yaml` owns package and gate expectations; `experiment_plan.yaml` records the review design; `data/claim_ledger.yaml` records the provenance tier and fixture/synthetic status of manuscript-facing claims.